

Crisis Coordination Platform for Emergency Response (ResQ)

Project Proposal for UCS503P

Thapar Institute of Engineering and Technology

GROUP MEMBERS (3Q24):

Gurnoor Singh Marok	1024170226
Rishi Gupta	1024170043
Tamanna Chadha	1027170237
Kaif Ahmad	1024170255

1 Higher-order goal

This project aims to strengthen disaster and emergency-response coordination by reducing the delay between a crisis being reported and help actually reaching the affected person. While the underlying problem — coordinating independent actors under uncertainty — is broader than any single deployment, the immediate engineering objective is to translate that coordination problem into a measurable, deployable software solution: a platform that connects affected people, volunteers, NGOs, resource providers, and administrators around a shared, real-time view of who needs help and who is providing it.

2 Time-to-value

- We will deliver meaningful increments quickly by phasing the build into explicit sessions, each producing a working, demo able slice rather than one large integrated release.
- We will prioritize fast feedback over completeness: Phase 1 (request submission, registration, manual assignment, map view) must stand on its own as a complete, demo able product before real-time coordination or matching logic is added.
- Success is defined by what can be built and self-tested in an agentic IDE loop (plan → code → browser-test) within each session, then extended in the next iteration rather than attempted all at once.

3 Problem Statement

During a local emergency, the hard problem is not collecting a report — that is a solved, shallow problem (a form and a map pin). The hard problem is coordinating independent actors who each hold partial information and sometimes conflicting incentives: affected people, volunteers, NGOs, resource providers, and administrators. Common failure modes include:

- Duplicated or wasted effort: two NGOs race to the same request while another goes unanswered, because neither can see the other's claim in real time.

- Fragmented visibility: each role (affected person, volunteer, NGO, admin) works from its own dashboard, and there is no single mechanism that keeps all four in sync as a request's status changes.
- Unmatched partial resources: a single need (e.g., transporting eight people) often has to be met by combining several partial contributions from different providers, which plain CRUD systems do not model well.
- No accountability loop: requests are frequently marked “resolved” by a responder without any confirmation from the affected person that help actually arrived, which undermines trust in the system.

Why this matters now (contextual relevance): In a crisis, response capacity is already scarce; every request answered twice, or not seen in time by the nearest available responder, is capacity the system has wasted.

4 Proposed Solution

4.1 Overview

- Emergency reporting portal: affected people or bystanders submit a request with category, description, and location; location is shown publicly only in approximate/fuzzy form.
- Verification and status workflow: a request moves through a defined lifecycle rather than a single open/closed flag.
- Cross-role real-time propagation: when one actor accepts or updates a request, every relevant dashboard (affected person, NGO, responder, admin) updates without a manual refresh.
- Resource and mission matching: volunteers, NGOs, and resource providers can be matched to requests individually, or combined into a mission when one need requires several partial contributions.
- Trust and accountability: a request cannot be closed until the affected person confirms that help was actually received, and exact contact/location details are revealed only after a claim is accepted.

4.2 Core workflow

1. An affected person (or bystander) submits an emergency request and receives a request ID and a confirmation immediately.
2. The request enters the system as Unverified; an admin or automated check moves it to Verified once basic completeness/plausibility checks pass.
3. A volunteer, NGO, or resource provider claims the request (or is matched to it); the request moves to In Progress and is simultaneously removed from other actors' available-request lists.
4. The responder marks work as complete, moving the request to Resolved; the affected person then confirms resolution, which is what actually moves the request to Closed.
5. Throughout, status changes are written to the database first and then broadcast over a real-time channel, so all dashboards reflect the same source of truth.

4.3 Operational constraints for an educational, capstone setting

- Keep the stack mainstream and well-documented (React/Vite, Node/Express, MongoDB/Mongoose) so it can be built and self-tested reliably, including with agentic coding tools.
- Avoid dependence on paid infrastructure; use free/self-hostable services (MongoDB + Mongoose, Leaflet + OpenStreetMap instead of a billed maps API, free-tier hosting for frontend and backend).
- Treat AI-based classification as strictly optional and pluggable, so the core coordination system continues to work if the AI module is removed or its quota is exhausted.

5 Solution Approach

This is an engineering course project, so the emphasis is on correct, scalable coordination and workflow logic rather than on novel matching-algorithm research.

5.1 Matching and allocation assistance

- Matching will be rule-based, not learned: a weighted score combining location proximity, suitability, availability, and capacity, without claiming state-of-the-art performance.
- The score normalizes each factor onto a comparable scale and explicitly handles missing data (e.g., a volunteer with no declared availability window) rather than silently failing.
- Matching suggestions are shown to staff/responders for acceptance; the system never auto-assigns in a way that could double-book a single resource to multiple requests simultaneously.
- A learned re-ranking model is treated as an optional later-phase stretch goal, not a dependency of the core workflow.

5.2 Web architecture

- Frontend: a responsive React (Vite) + TypeScript application with role-specific dashboards (affected person, volunteer, NGO, resource provider, admin).
- Backend API: a Node.js + Express service handling authentication, request submission, status transitions, and assignment/matching logic, exposed as a stateless REST API.
- Real-time layer: Socket.IO propagates every state change to the affected dashboards, with the database always treated as the single source of truth and sockets used purely as a push layer.
- Database: MongoDB + Mongoose

5.3 CI/CD and fast delivery enabler

- Automatically build and run unit/integration tests (Vitest/Jest, Supertest) on every change.
- Use Playwright end-to-end tests for the multi-role scenarios (e.g., one NGO claiming a request while a second NGO's dashboard updates live) that are the hardest part of this system to get right.
- Produce repeatable deployment artifacts to a free-tier staging environment, enabling frequent, low-risk releases across the phased build sessions.

6 Evaluation Criterion

6.1 Primary evaluation metric

Time-to-Match (TTM): the time between a request being marked Verified and it being claimed or matched to a responder.

- Target: median TTM ≤ 15 minutes during pilot testing.
- Attribution: measured directly from database timestamps (verification time vs. first assignment/claim event).

6.2 Secondary evaluation metrics

- Propagation latency: time between a state change being written to the database and it appearing on all other affected dashboards; target under 2 seconds, with zero missed updates in pilot testing.
- Double-booking rate: percentage of resources matched to more than one active request at the same time; target 0% in pilot testing.
- Confirmation rate: percentage of Resolved requests that reach Closed via affected-person confirmation, rather than being left unconfirmed.
- Reliability: uptime of staging/production for submission, claiming, and status updates during pilot hours (e.g., $\geq 99\%$).

6.3 Pilot validation plan

- Recruit a small simulated pilot: 10–20 users playing affected persons/volunteers and 3–5 users playing NGO/admin roles, if real NGO participation is not available.
- Run the core demo scenario end-to-end (submit $\rightarrow$ verify $\rightarrow$ claim $\rightarrow$ cross-dashboard update $\rightarrow$ resolve $\rightarrow$ confirm) and log timestamps for every metric above.
- Compare observed metrics against the targets in 6.1/6.2 and use the gaps to prioritize the next build session.

7 Scalability

1. Scalable (theoretically): the system should support growth in the number of concurrent requests and dashboards by:
 - decoupling the frontend and backend into independent, stateless-API-driven applications,
 - indexing and paginating the queries that back the request lists and mission-fulfillment views,
 - keeping the database, not the socket layer, as the single source of truth so horizontal scaling of the real-time layer does not risk inconsistent state.
2. Operationally deployable (practically): the system should run entirely on free-tier infrastructure by:
 - using MongoDB Atlas or another suitable free-tier MongoDB deployment option,
 - deploying the frontend and backend independently on free-tier platforms (e.g., Vercel/Netlify and Render/Railway),
 - running the AI classification module as a separable microservice that can be scaled, disabled, or swapped without touching the core system.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based, real-time project without building any of the following from scratch:

- Web framework for REST APIs and a componentized frontend (Express; React + Vite).

- Document database with a mature ODM (MongoDB via Mongoose).
- Real-time transport for cross-dashboard propagation (Socket.IO).
- Authentication approach for five distinct roles (hand-rolled JWT + bcrypt, no paid auth vendor required).
- Mapping and geocoding without a billed API (Leaflet + OpenStreetMap + Nominatim).
- Object storage for optional attachments (local storage for the pilot; Cloudflare R2 or Supabase Storage free tier for deployment).
- Test frameworks for unit, integration, and end-to-end coverage (Vitest/Jest, Supertest, Playwright).
- Optional LLM access for free-text urgency/category classification, kept as a separate, removable service.

We will use these mature, mainstream components deliberately, so effort goes into coordination correctness and workflow design rather than into building infrastructure that already exists.

9 Project scope and deliverables

9.1 Initial deliverable

- Request submission form with immediate request-ID generation, and location captured with a public/precise distinction.
- Volunteer, NGO, and resource-provider registration, with role-based access control.
- Manual admin assignment and the full status lifecycle (Unverified → Verified → In Progress → Resolved → Closed).
- Leaflet-based map view of open requests.
- CI: build, backend unit tests, and linting; CD to a single free-tier staging environment.

9.2 Subsequent deliverables

- Phase 2 — Coordination: Socket.IO wiring for the four-dashboard real-time propagation scenario, resource allocation tables, mission management, in-app messaging, and the resolution/confirmation flow.
- Phase 3 — Advanced: the rule-based matching engine (Section 5.1), an analytics/admin dashboard, and the optional AI classification microservice.
- Phase 4 — Hardening and demo: Playwright end-to-end scripts for the full multi-role scenario, convincing seed data, and deployment to free-tier hosting for the live submission.

10 Risks and mitigations

- Scope creep via entity count: five roles, multiple dashboards, missions, matching, and optional AI is a lot of surface area — mitigate by treating the phase gating in Section 9 as load-bearing, not decorative, and ensuring Phase 1 is a complete product on its own.
- The matching algorithm looks trivial but is not: normalizing distance against capacity, handling missing availability data, and preventing a single resource being suggested to multiple requests at once all need explicit design and, mitigate with database-level transaction locks around claim/assignment.

- Real-time correctness is harder than real-time plumbing: every state mutation must follow a strict “write to the database, then broadcast” discipline; mitigate by centralizing state transitions through one code path rather than duplicating broadcast logic per handler.
- Location and contact privacy: affected-person details cannot be broadcast to every volunteer; mitigate by showing only an approximate location publicly and revealing precise location/contact only after a claim is accepted.
- Mission-fulfillment tracking is the hardest data-modeling piece: reliably summing partial allocations against a per-mission, per-category target requires an explicit quantity-fulfilled schema decided before development begins, not retrofitted later.

11 Summary

The implementation will use the MERN stack (MongoDB, Express.js, React.js, and Node.js), supplemented by Socket.IO, Leaflet/OpenStreetMap, JWT/bcrypt, and free/open-source or free-tier services to keep the project cost-effective and suitable for an educational deployment.

This project proposes a deployable, real-time coordination platform to reduce the time between an emergency being reported and help actually being delivered in a resource-constrained response setting. It meets the course criteria by emphasizing time-to-value through explicitly phased build sessions, using CI/CD to support frequent and safe releases, and defining measurable evaluation criteria — principally time-to-match and cross-dashboard propagation latency. It remains focused on engineering coordination correctness, real-time consistency, and scalability readiness, while treating AI-assisted classification as an optional, decoupled enhancement rather than a core dependency.